

3. A Importância da Engenharia de Requisitos

“Se você não identifica os requisitos certos, não importa
quão bem você execute o resto do projeto.”

Karl Wieggers “Não há nada tão inútil quanto fazer eficientemente
o que não deveria ser feito.”

Peter Drucker

Este capítulo apresenta elementos que justificam os investimentos na Engenharia de Requisitos, principalmente em função dos desperdícios que sua aplicação pode evitar e do maior alinhamento do projeto às necessidades de negócio que ela promove. Apresentam-se evidências de como a negligência em atividades pouco onerosas provocam prejuízos monumentais e de que a negligência na definição do “contrato” entre quem tem necessidades de negócio e quem provê soluções em software está mais próxima a uma regra que a uma exceção.

3.1. Motivação para a Engenharia de Requisitos

A Engenharia de Requisitos é uma das disciplinas fundamentais à engenharia de software e que provê informações para maioria das demais disciplinas. Isso por si só já seria motivo para ressaltar sua importância.

Este capítulo apresenta os resultados de pesquisas que embasam de forma quantitativa esse ponto. Em resumo, o que se busca demonstrar é que negligenciar a disciplina de requisitos traz como consequências: atrasos no cronograma e custo adicional, nível alto de defeitos no software entregue e, principalmente, a entrega de um software que não satisfaz plenamente as necessidades do cliente.

É possível que o leitor discorde de alguns dos números apresentados, afinal são trabalhos de pesquisa sobre dados de amostras de projetos e empresas, não de todo o universo. Porém, não se deve ficar preso aos números em si, mas aos fenômenos destacados. Certamente o leitor com alguma experiência em projetos perceberá que tais fenômenos estão presentes também em sua realidade, variando apenas na intensidade apontada pelas pesquisas.

Apontam-se também algumas constatações que demonstram o quanto a ER é negligenciada pelo mercado e pelo meio acadêmico na formação de novos profissionais para desenvolvimento de software.

3.2. Impactos negativos da falha em requisitos

Antes de abordar os resultados de pesquisas que reforçam a importância da Engenharia de Requisitos, ilustram-se a seguir alguns casos famosos de falhas em software ou em projetos relacionadas de alguma forma à deficiência no exercício da Engenharia de Requisitos. Isso também ajudará a sensibilizar o leitor sobre sua importância. É possível que você tenha conhecimento de outras histórias – neste caso, fale com os autores. Interessa-nos conhecer mais “causos” sobre o tema.

3.2.1. Sonda espacial Mars Climate Orbiter

A *Mars Climate Orbiter* (MCO) foi uma sonda espacial norte-americana cujo objetivo primário era o estudo do clima marciano. Foi lançada em dezembro de 1998, alcançando Marte nove meses e meio depois. Porém, ao

entrar na órbita de Marte, a MCO foi destruída na atmosfera devido a um erro de cálculo nessa manobra (Figura 3.1).

Somente a perda da sonda resultou num prejuízo de US\$ 125 milhões à NASA (agência espacial norte-americana). Isso sem considerar ainda os custos de desenvolvimento do foguete para o lançamento, do próprio lançamento da sonda e da operação da missão.

Uma investigação apurou que a causa primária do erro na trajetória de entrada em Marte da sonda foi um software desenvolvido por um fornecedor que produzia resultados no sistema imperial britânico (*pounds-seconds*). Isso estava em desacordo com a exigência da NASA, que esperava as medidas no sistema métrico universal (*newton-seconds*). Quando os resultados foram usados por outro software (este desenvolvido pela própria NASA), isso resultou no erro do cálculo da rota de entrada na atmosfera, causando sua destruição.

Não se sabe afirmar se a NASA forneceu as especificações detalhadas ao fornecedor (o que neste caso remete a um erro de especificação devido à inconsistência na interface entre os dois softwares) ou se o fornecedor participou do desenvolvimento dos requisitos para o software (neste caso pode ter havido uma falha de levantamento).

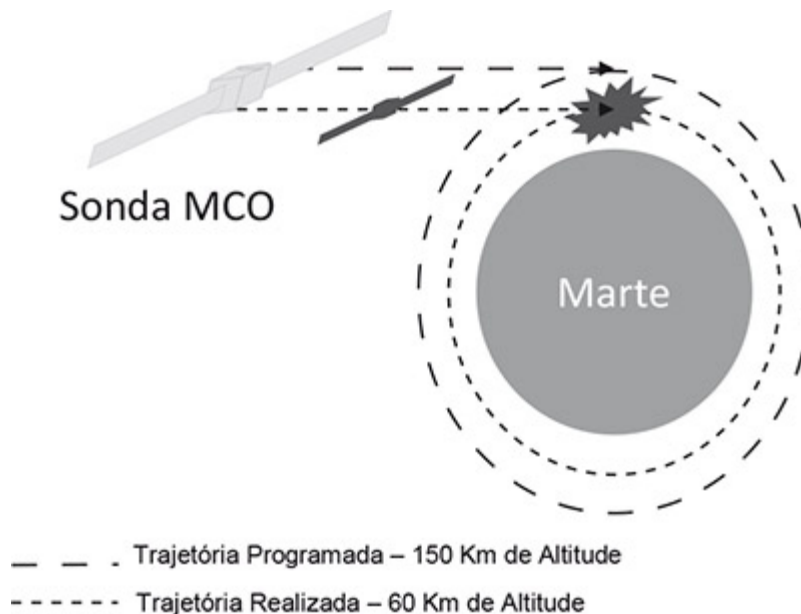


Figura 3.1. O controle da Terra enviou instruções de correção de rota para o MCO na unidade *pounds-seconds* e o software da MCO esperava dados na unidade

newton- -seconds. Isso antecipou sua entrada na atmosfera de Marte, causando sua destruição.

3.2.2. Míssil antibalístico *Patriot*

Durante a Guerra do Golfo na década de 1990, a coalizão de forças liderada pelos EUA usou um sistema de defesa com mísseis antibalísticos *Patriot* (Figura 3.2). Em 25 de fevereiro de 1991 este sistema falhou ao não interceptar um míssil *Scud* lançado pelo Iraque. O míssil iraquiano matou 28 militares americanos e feriu outros 98. Posteriormente descobriu-se que a falha estava no software do míssil.

O software original que calculava a trajetória do *Patriot* utilizava dados dos sinais de radar e tinha que trabalhar com precisão de frações de segundo. Para lidar com mísseis mais modernos de alta velocidade foi criada uma sub-rotina para obter de forma mais precisa (mais casas decimais) as informações do relógio do sistema. Porém, esta sub-rotina não foi utilizada em todas as partes necessárias do software. A consequência disso é que o acúmulo de falhas de precisão, depois de certo tempo, tornava o sistema de interceptação do míssil ineficaz. Não foi apenas uma falha de programação, mas uma falha de avaliação do impacto da mudança.

Aparentemente essa deficiência do sistema era conhecida e uma solução de contorno bastante comum em software foi adotada: “reiniciar o sistema sempre que possível” para zerar os acúmulos de imprecisão. Lembre-se que isso ocorreu na década de 1990, e atualizar o software dos equipamentos em campo de guerra não seria trivial. Infelizmente o termo “sempre que possível” da solução dada é vago. Quando o míssil iraquiano foi lançado, o sistema do *Patriot* estava rodando sem ser reiniciado há mais de cem horas, e o acúmulo de imprecisões havia deixado o sistema defasado em um terço de segundo, o que impossibilitou a interceptação.



Figura 3.2. O software responsável pela trajetória do míssil *Patriot* tinha um defeito que causava atraso no seu relógio. A solução de contorno especificada foi reiniciar o sistema “sempre que possível” (crédito: Shutterstock).

3.2.3. Foguete Ariane 501

Em 4 de junho de 1996, o foguete Ariane 501 (da agência espacial europeia) foi lançado da base Kourou, na Guiana Francesa, e aproximadamente 37 segundos após a decolagem desviou-se bruscamente do curso previsto, partiu-se e explodiu (Figura 3.3). Este foi o primeiro lançamento da série Ariane 5, consumiu dez anos de desenvolvimento e teve um custo de US\$ 7 bilhões. A perda do foguete e sua carga somou um prejuízo de mais de US\$ 370 milhões.

A comissão de investigação apontou que a causa da falha foi a perda total de informações de orientação do foguete após a sequência de ignição principal. Essa perda de informação foi provocada por erros de

especificação e *design* do software do sistema de referência inercial. Este software foi reutilizado da série Ariane 4. Porém, os foguetes dessas séries possuíam algumas características distintas, enquanto o Ariane 5 foi projetado para levar mais carga, o que implica em padrões distintos de trajetória e velocidade. O erro foi não adaptar o software do Ariane 4 aos requisitos distintos do Ariane 5.



Figura 3.3. O foguete Ariane 501 explodiu 37 segundos após lançamento por falha na especificação do software do seu sistema de referência inercial. Veja em: https://youtu.be/gp_D8r-2hwk (crédito: Shutterstock).

3.2.4. HAL 9000

Este é um exemplo da ficção. Na obra “2001: uma odisseia no espaço”, o computador de bordo da espaçonave, chamado HAL 9000, tenta matar toda a tripulação. Na obra seguinte, “2010: odisseia dois”, descobre-se que esse comportamento do computador ocorreu porque ele foi programado para cumprir dois requisitos conflitantes: divulgar plenamente suas informações e manter segredo para a tripulação do verdadeiro objetivo da missão.

Portanto, a solução encontrada por HAL para cumprir os dois requisitos foi eliminar a tripulação.

3.2.5. Arquivo virtual do FBI

No início do ano 2000 o FBI (a polícia federal norte-americana) iniciou o desenvolvimento de um software chamado *Virtual Case File* (VCF), uma espécie de arquivo virtual dos processos de investigação conduzidos pelo FBI que permitiria compartilhamento de informações de diferentes casos entre os agentes. O planejamento original previa uma duração de três anos para o projeto.

Após os ataques terroristas de 11 de setembro de 2001, o FBI recebeu fortes críticas por não ter antecipado o ataque, dadas as evidências que foram descobertas depois. A falha foi não ter conseguido estabelecer a relação entre as várias evidências disponíveis. O VCF, que ajudaria em parte nesse objetivo, passou a ser uma das prioridades máximas do FBI: seus requisitos aumentaram, assim como prazos e custos.

Porém, depois de cinco anos de desenvolvimento e de custos incorridos de US\$ 170 milhões, o projeto foi abortado com o sistema ainda em desenvolvimento. Investigações posteriores constataram várias causas para o fracasso do projeto, dentre elas: ➤

- Mudanças frequentes em requisitos. O fornecedor contratado para o desenvolvimento alegou que o FBI adotou a tática de tentativa e erro em várias decisões e que a filosofia dos responsáveis era “vou saber o que eu quero depois de ver o produto”.



- Alta rotatividade da gerência (o que também contribuiu para as mudanças frequentes em requisitos).



- Aumento não controlado do escopo, com requisitos sendo acrescentados mesmo com o projeto já atrasado.

3.3. Uma tentativa de melhoria de processo

Ganho de produtividade é o sonho de toda empresa, e com as que desenvolvem software não é diferente. Visando isso, a diretoria de Tecnologia da Informação de certa empresa investiu fortemente na aquisição de ferramentas para desenvolvimento de software do topo do mercado e também na capacitação de sua equipe técnica nessas ferramentas. Para alguns, investiu ainda na certificação profissional das ferramentas. Depois de certo tempo, ficou perceptível que a equipe passou a trabalhar com mais desenvoltura com as ferramentas e o quão ágil era agora para criar coisas novas. A empresa estava orgulhosa de sua “tropa de elite” para desenvolvimento de software.

No entanto, para as demais áreas clientes não houve benefícios perceptíveis. A insatisfação com os projetos entregues pela TI ainda era alta. Os prazos e orçamentos continuavam a ser excedidos. Uma investigação foi feita para tentar descobrir as causas dos problemas. Uma primeira constatação foi descartar o ferramental e o pessoal não qualificado tecnicamente como causa do problema. Na visão dos clientes da TI, o problema era que ela não conseguia entender suas necessidades e sempre entregava algo que não lhes atendia. Na visão dos desenvolvedores, o problema era dos clientes, que nunca sabiam ao certo o que desejavam, o que os obrigava a refazer um mesmo trabalho várias vezes.

Em resumo, velocidade não era o problema, pois a equipe técnica conseguia produzir com rapidez. O problema era de direção: a equipe não conseguia produzir a coisa certa. E a direção correta era descoberta por tentativa e erro. Cada tentativa era até rápida para se produzir, mas o ciclo todo até chegar à tentativa definitiva era o que interessava ao cliente – e continuava demorado. O trabalho da equipe técnica podia ser descrito em uma frase: “entra lixo, sai lixo. Mas, uau, com que velocidade!”.

Faltava a essa empresa a atenção devida à Engenharia de Requisitos. Ela é a responsável por construir essa ponte entre as necessidades do cliente e uma solução que o satisfaça de forma eficaz.

3.3.1. Principal motivo para o fracasso de projetos

O *Project Management Institute* (PMI) apresenta uma constatação preocupante: 47% dos projetos que fracassam são causados por uma deficiência no exercício da Engenharia de Requisitos (PMI, 2014). Esse estudo não é restrito a projetos de software, mas a projetos de forma geral. É possível que o resultado seja ainda pior para o contexto de projetos de software.

Quando um projeto de engenharia civil falha na Engenharia de Requisitos, é mais difícil ocultar: uma construção, seja ela uma ponte, um edifício, uma usina ou uma estrada, é visível, e, se for refeita, ainda que parcialmente, o fato será percebido por muitos. Para um projeto de software, as falhas de requisitos também vão gerar retrabalho, porém isso não costuma ser tão visível. Se um programa precisar ser reescrito, possivelmente somente os membros da equipe técnica irão ter ciência disso.

Somente o fato de ser o maior motivo isolado de fracassos em projetos já justificaria a importância da Engenharia de Requisitos. E isso deveria resultar em um investimento adequado em termos de planejamento para melhorar o seu exercício. Porém, o mesmo estudo constata que apenas 20% das organizações pesquisadas possuem alta maturidade com a Engenharia de Requisitos. Esse mesmo estudo também revela que organizações com baixo desempenho na Engenharia de Requisitos desperdiçam quase dez vezes mais dinheiro em projetos que as de alto desempenho. Ou seja, enquanto as organizações não evoluírem sua maturidade com requisitos, a possibilidade de fracassos nos projetos continuará sendo significativa.

E por que deficiência na Engenharia de Requisitos potencializa a chance de fracasso do projeto? Por vários motivos, como os descritos a seguir.



Estimativas de custo com um nível de incerteza compatível com seu propósito têm como matéria-prima requisitos de qualidade. Aliás, requisitos de qualidade podem mostrar que o projeto não é viável, não justificando seu investimento. Nem todas as necessidades de negócio devem ser atendidas a qualquer custo e nem todas aquelas que originalmente se pretende atender por meio de um novo produto de software precisam ser resolvidas dessa forma. Muitos projetos

fracassam porque são iniciados com a premissa de custo subestimada, embasada em um escopo que desconsidera áreas funcionais, macroprocessos e pessoas que são afetadas pelo software em desenvolvimento.



A entrega, ao final do projeto, de um produto que não satisfaça o cliente. Ainda que o projeto tenha sido concluído no prazo e no orçamento previsto, este será um fracasso.



O projeto incorporar ao produto requisitos que não deveriam estar no escopo. São requisitos que consomem recursos do projeto e concorrem com requisitos necessários. Promovem o risco e comprometem a capacidade de entregar um produto que atenda às necessidades de negócio que motivaram seu desenvolvimento, observando as premissas de custo e tempo que o justificaram em primeiro lugar.



O produto ser entregue sem cumprir com requisitos necessários e que sequer foram identificados. Isso pode resultar em um produto inútil ou em retrabalho para adequá-lo aos requisitos essenciais e que não foram descobertos.



Falhas na comunicação dos requisitos ao longo do projeto, potencializando a sua não compreensão e, como resultado, a entrega de um produto deficiente cuja adequação exige retrabalho.



Há requisitos que apenas são descobertos com o uso de protótipos; principalmente de usabilidade. Contudo, há mudanças que são falta de cuidado da equipe de desenvolvimento em entender e desenvolver as necessidades do cliente. São mudanças que desperdiçam recursos e que poderiam ser evitadas se as necessidades fossem mais bem compreendidas inicialmente e se tivesse existido cuidado na sua comunicação ao longo do desenvolvimento do produto; por exemplo, com uma especificação adequada a esse propósito.

3.3.2. Uma das principais causas de defeitos em software

Para o público leigo, em geral, um produto é de qualidade quando não apresenta defeitos. O mesmo vale para software. O critério para avaliação de qualidade mais comumente utilizado, não só pelos usuários do software como também pela equipe técnica de desenvolvimento, são os defeitos percebidos no produto.

O que se espera, portanto, para que seja possível considerar um software como tendo qualidade? Seguindo a definição de qualidade da ISO 9000, conclui-se que software de qualidade é aquele cujas características presentes atendem aos seus requisitos.

No entanto, é frequente a queixa dos usuários de software sobre falhas no seu funcionamento. Os defeitos podem se originar em qualquer fase do ciclo de vida do projeto. O mais comum é que erros sejam inseridos durante a construção do software. Ou seja, o produto construído tem um comportamento diferente do especificado.

Porém, uma parte significativa dos defeitos tem origem nos requisitos. Como assim? Qualidade não é cumprir com requisitos? Como seria possível o requisito ter defeito? Se o software funciona de acordo com o requisito, isso não deveria ser considerado defeito!

Se isso parece confuso, lembre-se que a definição de requisito apresentada no Capítulo 2 possui três partes. Faz parte da Engenharia de Requisitos identificar “as capacidades necessárias por um usuário para resolver um

problema” (parte 1 da definição) e criar uma “representação documentada” (parte 3 da definição).

Qualquer falha em identificar essas capacidades, ou qualquer falha em documentar essas capacidades, traduz-se então em um defeito introduzido pelo exercício da Engenharia de Requisitos, pois resultará em produto com comportamento indesejado pelo cliente.

Observe que prover documentação em excesso é uma falha em documentar; prover documentação descrevendo algo que apenas se poderá confirmar após prototipação é uma falha em documentar. Ou seja, avaliar se houve uma falha na documentação requer avaliar os fatores discutidos no Capítulo 2.

Logo, não se pode dizer que o software é de qualidade simplesmente porque é aderente a uma especificação, se esta especificação não representa corretamente as necessidades do usuário. Os desenvolvedores implementarão tudo o que foi dito na especificação de requisitos, ou o que conseguiram entender dela. Ocorre que tal especificação nem sempre reflete fielmente as necessidades dos usuários. Como evitar esse problema? Veja os tópicos de verificação e validação no Capítulo 8.

Capers Jones (2013) apresenta que, nos EUA, um em cada cinco defeitos em potencial são originados de requisitos, ou seja, 20% do total. Dean Leffingwell (1997) argumenta que, em projetos mais complexos, erros em requisitos são os mais comuns. Pohl (2011) afirma que erros em requisitos podem corresponder a até 60% do total de erros no projeto, isso considerando também como erro requisitos que faltaram ser implementados no produto. E este tipo de erro só é detectado em etapas mais avançadas do trabalho, não raro na homologação do produto.

Na Tabela 3.1, Jones (2012) apresenta os defeitos em potencial em software, segregados por fonte do defeito e estratificados por ordem de grandeza do sistema. O tamanho do software está expresso em pontos de função, que é uma unidade de medida de software na sua perspectiva funcional (VAZQUEZ, 2013).

Tabela 3.1. Defeitos em potencial por tamanho de sistema e origem.

Origem do defeito	Tamanho do sistema (em pontos de função – PF)		

	100	1.000	10.000
	Defeitos potenciais (<i>bugs</i> /PF)		
Requisitos	0,75	1,00	1,25
Arquitetura	0,10	0,25	0,50
Projeto (<i>design</i>)	1,00	1,25	1,50
Código-fonte	1,70	1,75	2,00
Material de testes	1,50	1,85	2,00
Documentação	0,65	0,70	0,75
Base de dados	2,00	2,75	3,00
Website	1,50	1,75	2,00
Total	9,20	11,30	13,00

Na Tabela 3.1 os defeitos em requisitos têm incidência menor que os dos outros autores citados. Os defeitos com origem no código-fonte, base de dados ou website têm incidência maior – no entanto, são os defeitos mais fáceis de eliminar. Já os defeitos originados em requisitos são mais difíceis de eliminar com os métodos tradicionais: teste e análise estática. Isso porque, uma vez que uma especificação tenha sido aprovada com um requisito defeituoso, os casos de testes elaborados a partir dessa especificação tenderão apenas a confirmar seu comportamento equivocado.

O mesmo autor também destaca que as mudanças em requisitos tendem a ter uma densidade de defeitos maior que os requisitos originais, o que não chega a ser tanto uma surpresa, pois as mudanças costumam ser tratadas em geral de forma apressada. Isso também faz com que esses defeitos sejam de mais difícil remoção, justamente por muitas vezes “pularem” etapas do controle de qualidade do projeto. A título de ilustração desse fenômeno, um crescimento de 10% no escopo do projeto implica em um aumento de 12% na quantidade de defeitos.

3.3.3. Custo do reparo de defeitos

A indústria de software tem um potencial muito grande para explorar ganho de eficiência. De acordo com Boehm (2001), de 40% a 50% do esforço em projetos de software são gastos em retrabalho para a correção de defeitos. Um dos fatores que contribui para essa ineficiência é que, ao se corrigir um defeito, há de 20% a 50% de chance de se introduzir outro defeito no software (BROOKS, 1995).

Boehm (2001) também aponta que o esforço para encontrar e corrigir um defeito após a entrega do software pode frequentemente custar com vezes mais do que corrigir este defeito enquanto ainda se está elaborando requisitos (Figura 3.4). E, como vimos nos casos citados no início do capítulo, erros detectados com o software em produção podem causar prejuízos várias ordens de grandeza maiores que o próprio custo de correção do defeito.

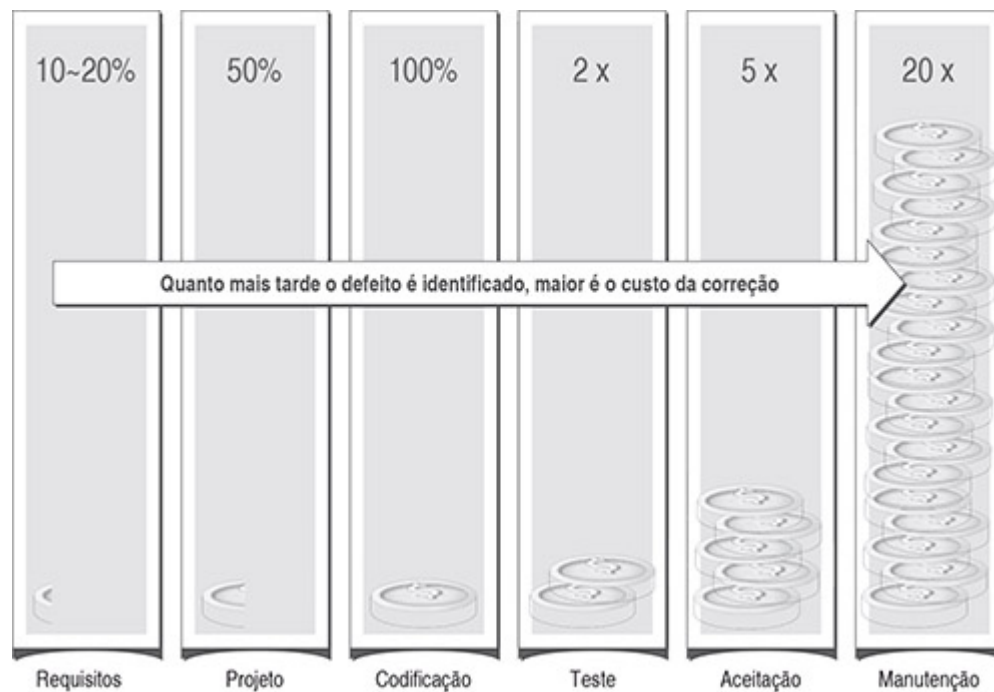


Figura 3.4. Custo relativo para correção de defeitos conforme o ponto onde é identificado.

Pohl (2011) argumenta ainda que a maior parte dos erros originados em requisitos é detectada em fases adiantadas do projeto. Isso é coerente com a experiência dos autores, acrescentando que uma parte significativa desses erros é identificada quando da homologação ou validação pelo cliente.

A meta deveria ser fazer certo da primeira vez. Logo, minimizar erros nas etapas iniciais do projeto é onde se consegue mitigar com mais intensidade o retrabalho. O método de tentativa e erro, além de ser mais caro e demorado, gera insatisfação dos clientes. Logo, evoluir na qualidade do trabalho de requisitos impactará positivamente em custo, prazo e satisfação do projeto.

3.4. Especificações de requisitos como um ativo

Em muitas empresas o entendimento quanto ao funcionamento de um sistema legado depende de uma investigação de seu código-fonte. E o código-fonte é expresso em uma linguagem de programação de computadores, passível de compreensão quase sempre apenas por desenvolvedores, e muitas vezes somente por aqueles especializados naquela linguagem específica. Dependendo de um programador ler código-fonte para saber como o software funciona representa uma sobrecarga de trabalho significativa ao negócio.

Além disso, há riscos tanto na tradução do código-fonte para o entendimento do desenvolvedor quanto na transmissão desse entendimento do desenvolvedor aos responsáveis pela gestão do negócio. Também há um esforço analítico relevante do desenvolvedor, para além da tradução, juntar os diversos componentes de implementação nos quais uma única função no negócio se subdivide. Esse tipo de cenário representa uma sobrecarga muito grande às atividades de manutenção de software. Ou seja, mexer em um produto em que não se sabe direito como funciona, além de ser mais demorado, possui mais chance de erros.

O trabalho de requisitos envolve identificar, criar e estruturar o conhecimento. E, em tese, todo esse conhecimento pode ser guardado na cabeça de alguém. Porém, a memória humana é falha e as pessoas são perecíveis. Logo, é importante que esse conhecimento seja persistido (documentado). Neste caso, as especificações de requisitos funcionam como um ativo valioso na organização: representam esse conhecimento que foi elaborado. Elas ajudam a minimizar o impacto de rotatividade de

pessoal na equipe do projeto. Toda vez que alguém sai da equipe, uma parte do conhecimento se perde. No entanto, se a especificação de requisitos existe e possui um nível de detalhe adequado, tal perda é atenuada.

3.5. Reflexão: onde a indústria de software investe energia

Pelo exposto até este ponto, já se percebe o quão importante é a Engenharia de Requisitos. O próprio fato de você ser leitor deste livro já pressupõe que você julga o tema importante. Porém, cabe refletir um pouco sobre a relevância que a indústria e a academia dão a esse tema.

Do ponto de vista da formação acadêmica dos novos profissionais em desenvolvimento de software, é comum que a Engenharia de Requisitos seja apresentada como parte da disciplina de engenharia de software e não como disciplina específica. Isso resulta quase sempre em uma visão superficial do tema.

Para muitas empresas, o esforço de capacitação da equipe técnica é em sua maioria dedicada a ferramentas, às vezes até em ferramentas de gestão de requisitos. Mas sem uma base sólida dos conceitos fica difícil tirar o melhor proveito da ferramenta.

Para alguém interessado no tema, qual a oferta de livros sobre o assunto? Talvez o leitor já tenha feito essa pesquisa, mas, caso não tenha feito, recomendamos a experiência. Pesquise em uma livraria virtual qualquer livros sobre “requisito”. O resultado não será volumoso, contam-se nos dedos de uma só mão as opções disponíveis. Se incluirmos livros de engenharia de software, que tratam o tema em um capítulo, consegue-se aumentar, mas não muito, as opções. Se a pesquisa for por livros em inglês, daí há mais opções.

Mas compare o resultado dessa pesquisa com uma pesquisa sobre livros de uma determinada linguagem de programação (por exemplo: Java, .NET, PHP). Serão centenas de opções retornadas. Mesmo em inglês, a discrepância de livros sobre requisitos e sobre linguagem de programação é gigantesca.

Isso ajuda a perceber a inversão de valores praticada no mercado. Muitos vendem a ideia de que a ferramenta X ou Y resolverá seus problemas em desenvolvimento de software. É compreensível, pois há empresas vendendo esse ferramental, que movimenta altas somas. Porém, para aqueles que já têm um tempo de estrada no mercado, não é difícil perceber que o desenvolvimento de software vive de muitos modismos. As ondas passam, ferramentas vêm e vão. E os problemas continuam: projetos entregando produtos que não satisfazem o usuário.

Portanto, a maioria aprende sobre a Engenharia de Requisitos na prática, participando de projetos, percebendo o que funciona, o que não funciona (errando) e como superar essas dificuldades em futuros projetos. É um jeito caro de aprender.

3.6. Exercícios

1. Nos projetos de sua empresa que apresentam problemas ou fracassam, qual é o percentual deles que têm como uma das causas a deficiência na Engenharia de Requisitos? Cite alguns exemplos.
2. Qual é o percentual de esforço aproximado dedicado à disciplina de Engenharia de Requisitos nos projetos de sua empresa? Esse valor é adequado? Em que momento do projeto (primeiro, segundo ou último terço) esse esforço é mais intenso?
3. Do total de solicitações de mudanças de requisitos ao longo dos projetos em que você já participou, qual é o percentual aproximado de solicitações que poderiam ter sido evitadas se um trabalho melhor de requisito houvesse sido feito?
4. Qual é o percentual aproximado de defeitos detectados nos projetos em que participou que têm sua origem em uma falha do trabalho de requisitos?
5. É possível calcular o retorno sobre o investimento na melhoria das práticas de Engenharia de Requisitos de uma empresa? Explique.